

Agile

What is Agile methodology?

The Agile methodology is a collection of project management frameworks that break projects down into smaller phases. It relies on iterative cycles, allowing teams to adapt to changes and regularly refine their work. Agile is built on four main values from the [Agile Manifesto](#):

- **People over processes:** Teams work together closely, solving problems through open conversations.
- **Working solutions over detailed documentation:** Agile teams prioritize building and testing usable products over writing lengthy reports.
- **Customer collaboration over rigid contracts:** Agile encourages ongoing discussions with customers to adjust work as needed.
- **Adapting to change over following a strict plan:** Agile teams stay flexible, ready to adjust to new information, customer needs, or shifting priorities.

What are the benefits of Agile methodology?

Agile stands out because it focuses on delivering value quickly while staying **open to change**. This way of working brings several benefits, including:

- **Faster time to market:** Agile's short, focused cycles allow teams to release updates or products earlier. This helps businesses respond to market demands faster and stay ahead of competitors.
- **Better stakeholder involvement:** Regular collaboration ensures that products meet real customer needs, not assumptions. Feedback is gathered throughout the project, reducing the risk of delivering something that misses the mark.
- **Increased team productivity:** Teams break work into smaller tasks and review progress often, which helps them spot obstacles early, adjust priorities quickly, and improve processes over time. This leads to better outcomes and reduces wasted effort.

Challenge	Impact
Unclear scope and timelines	Hard to predict deadlines and costs
High stakeholder involvement	Can cause burnout and decision fatigue
Risk of lost details	Less documentation may lead to missing info
Harder to scale for large teams	Coordination becomes complex
Team discipline needed	Self-managing teams may lack focus and require additional managerial oversight

- **What are the disadvantages of Agile methodology?**
- Is Agile always the best choice? Not necessarily. Here are some of the most common drawbacks of Agile frameworks and how they can impact projects:

Agile methodologies vs. traditional approaches

Traditional project management approaches, like the [Waterfall methodology](#), follow a strict, step-by-step process where each phase must be completed before the next begins. This approach works best for projects with stable requirements.

Agile takes a different path. It breaks projects into smaller, repeatable phases where teams build, test, and adjust continuously. Customers are involved throughout, and teams can change direction anytime based on feedback or new priorities. This makes Agile better suited for fast-changing projects where adaptability and speed are key.

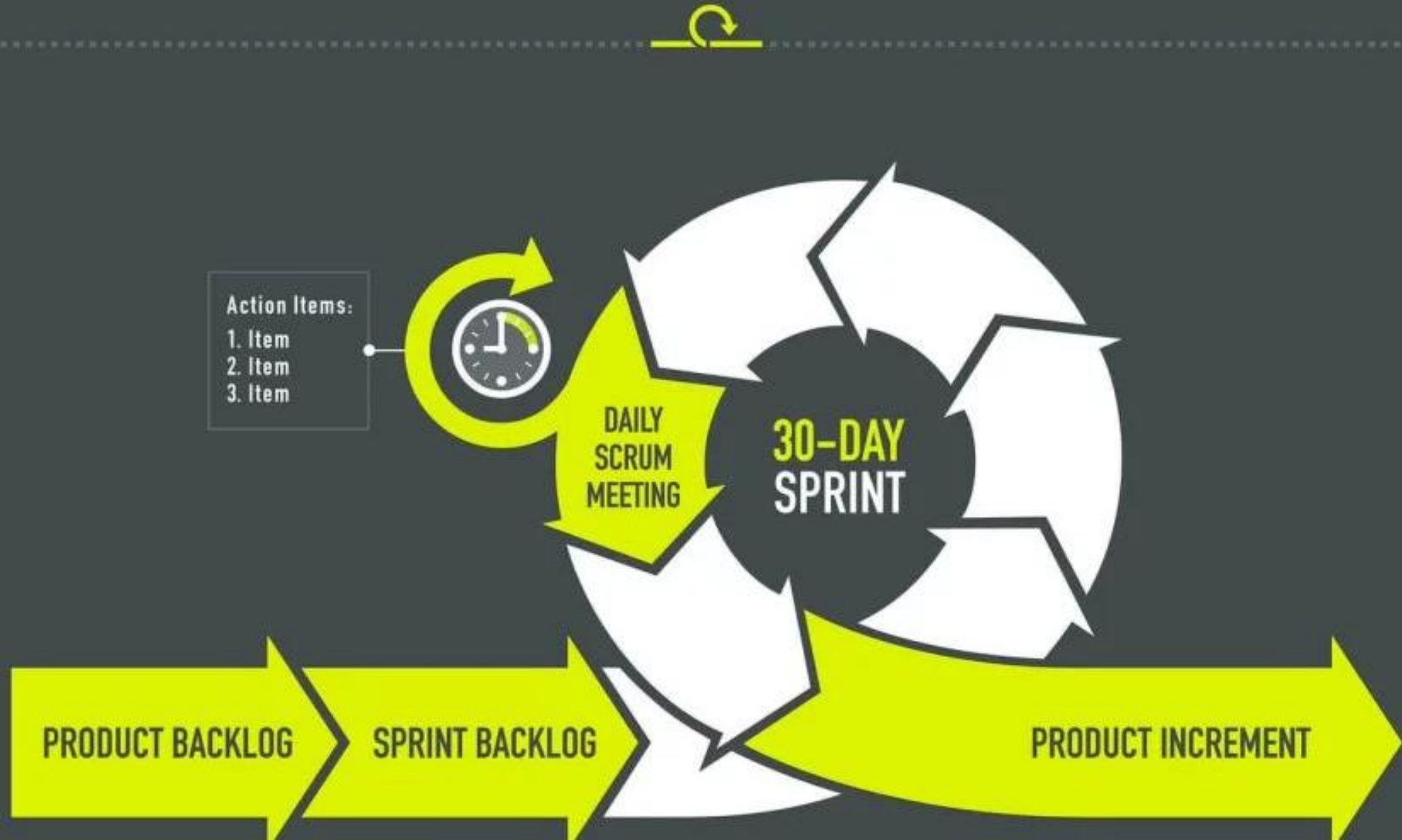
Types of Agile Models:

- Scrum
- FDD (Feature Driven Development)
- ASD (Adaptive Software Development)
- DSDM (Dynamic Systems Development Method)
- XP(Extreme Programming)
- Crystal
- Agile Modeling (AM)
- Kanban

THREE MAIN SCRUM ROLES



SCRUM DEVELOPMENT PROCESS



What is Scrum?

Scrum is a management framework that teams use to self-organize and work towards a common goal. It describes a set of meetings, tools, and roles for efficient project delivery. Much like a sports team practicing for a big match, Scrum practices allow teams to self-manage, learn from experience, and adapt to change. Software teams use Scrum to solve complex problems cost effectively and sustainably.

What is Scrum methodology?

Certain principles and values characterize Scrum methodology:

Scrum principles for project success

Transparency

Teams work in an environment where everyone is aware of the challenges that others might be experiencing. Regular face-to-face conversations between cross-functional team members and project owners prevent miscommunication and information bottlenecks.

Reflection

Frequent reflection points are built into the framework to allow team members to review their progress. Project managers use insights from these review meetings for estimation and future planning. As a result, projects can run more efficiently, within budget, and on schedule.

CONT'D

Adaptation

Team members can reprioritize tasks based on changing customer requirements. They decide which tasks to complete first and which to revisit in the future.

Scrum values for project teams

Scrum Teams follow five core values.

Commitment

Scrum Team members are committed to time-based tasks and goals and are dedicated to continuous improvement to find the best solution.

Courage

Scrum Teams show courage by asking open, challenging questions. They have honest and transparent discussions to arrive at the best solution.

Focus

During any given period, team members will work from a Product Backlog of tasks. They will focus on the selected tasks to provide deliverables within a limited time frame.

CONT'D

Openness

Scrum Team members are open to new ideas and opportunities that support individual learning and overall project quality.

Respect

Team members respect the project managers, each other, and the Scrum process. This culture of respect creates a spirit of mutual collaboration and cooperation within the team.

How does Scrum work?

Scrum is a framework that is easy to learn but difficult to become an expert in. The co-creators of scrum, Jeff Sutherland and Ken Schwaber, have explained the underlying concepts in The Scrum Guide. The guide gives a detailed overview of scrum processes and how to implement them effectively.

The essence of Scrum is a self-organizing team delivering customer value in a time-boxed period called a Sprint. Scrum defines artifacts, roles, and events associated with each Sprint. Let's look at each of these in detail.

What are Scrum artifacts?

Scrum Teams use tools called Scrum artifacts to solve problems and manage projects. Scrum artifacts provide critical planning and task information to team members and stakeholders. There are three primary artifacts:

Product Backlog

The Product Backlog is a dynamic list of features, requirements, enhancements, and fixes that must be completed for project success. It is essentially the team's to-do list, which is constantly revisited and reprioritized to adapt to market changes. The product owner maintains and updates the list, removing irrelevant items or adding new requests from customers.

Sprint Backlog

The Sprint Backlog is the list of items to be completed by the development team in the current Sprint cycle. Before each Sprint, the team chooses which items it will work on from the Product Backlog. A Sprint Backlog is flexible and can evolve during a Sprint.

Increment

The Increment is a step towards a goal or vision. It is the usable end product from a Sprint. Teams can adopt different methods to define and demonstrate their Sprint Goals. Despite the flexibility, the fundamental Sprint Goal—what the team wants to achieve from the current Sprint—can't be compromised.

For example, some teams choose to release something to their customers at the end of the Sprint, so their Sprint Goal would be completed once the software change is released. Other teams might work on completing a set of features that will be released together. In this case, the Sprint Goal would be completed when a feature is tested successfully.

What are Scrum roles?

A Scrum Team needs three specific roles: a Product Owner, Scrum leader, and development team.

Product Owner

The Product Owner focuses on ensuring the development team delivers the most value to the business. They understand and prioritize the changing needs of end users and customers. Effective product owners do the following:

Give the team clear guidance on which features to deliver next.

Bridge the gap between what the business wants and what the team understands.

Decide when and how frequently releases should happen.

Scrum leader

Scrum leaders are the champions for Scrum within their teams. They are accountable for the Scrum Team's effectiveness. They coach teams, Product Owners, and the business to improve its Scrum processes and optimize delivery. Scrum leaders are also responsible for doing the following:

- Schedule the resources needed for each Sprint.

- Facilitate other Sprint events and team meetings.

- Lead digital transformation within the team.

- Facilitate any team training when adopting new technologies.

- Communicate with external groups to solve any challenges the team might be facing as a whole.

Scrum development team

The Scrum Team consists of testers, designers, UX specialists, Ops engineers, and developers. Team members have different skill sets and cross-train each other, so no one person becomes a bottleneck in delivering work.

Jeff Bezos, the founder of Amazon, recommends the two-pizza rule when deciding team size: A team should be small enough to share two pizzas.

Scrum development teams do the following:

Work collaboratively to ensure a successful Sprint completion.

Champion sustainable development practices.

Self-organize and approach their projects with an evident we attitude.

Drive the planning and estimating for how much work they can complete for each Sprint.

What are Scrum events?

Scrum events or Scrum ceremonies are a set of sequential meetings that Scrum Teams perform regularly. Some Scrum events include the following:

Sprint Planning

In this event, the team estimates the work to be completed in the next Sprint. Members define Sprint Goals that are specific, measurable, and attainable. At the end of the planning meeting, every Scrum member knows how each Increment can be delivered in the Sprint.

Sprint

A Sprint is the actual time period when the Scrum Team works together to finish an Increment. Two weeks is the typical length for a Sprint but can vary depending on the needs of the project and the team. The more complex the work and the more unknowns, the shorter the Sprint should be.

Daily Scrum or stand-up

A Daily Scrum is a short meeting in which team members check in and plan for the day. They report on work completed and voice any challenges in meeting Sprint Goals. It is called a stand-up because it aims to keep the meeting as short as practical—like when everybody is standing.

Sprint Review

At the end of the Sprint, the team gets together for an informal session to review the work completed and showcase it to stakeholders. The Product Owner might also rework the Product Backlog based on the current Sprint.

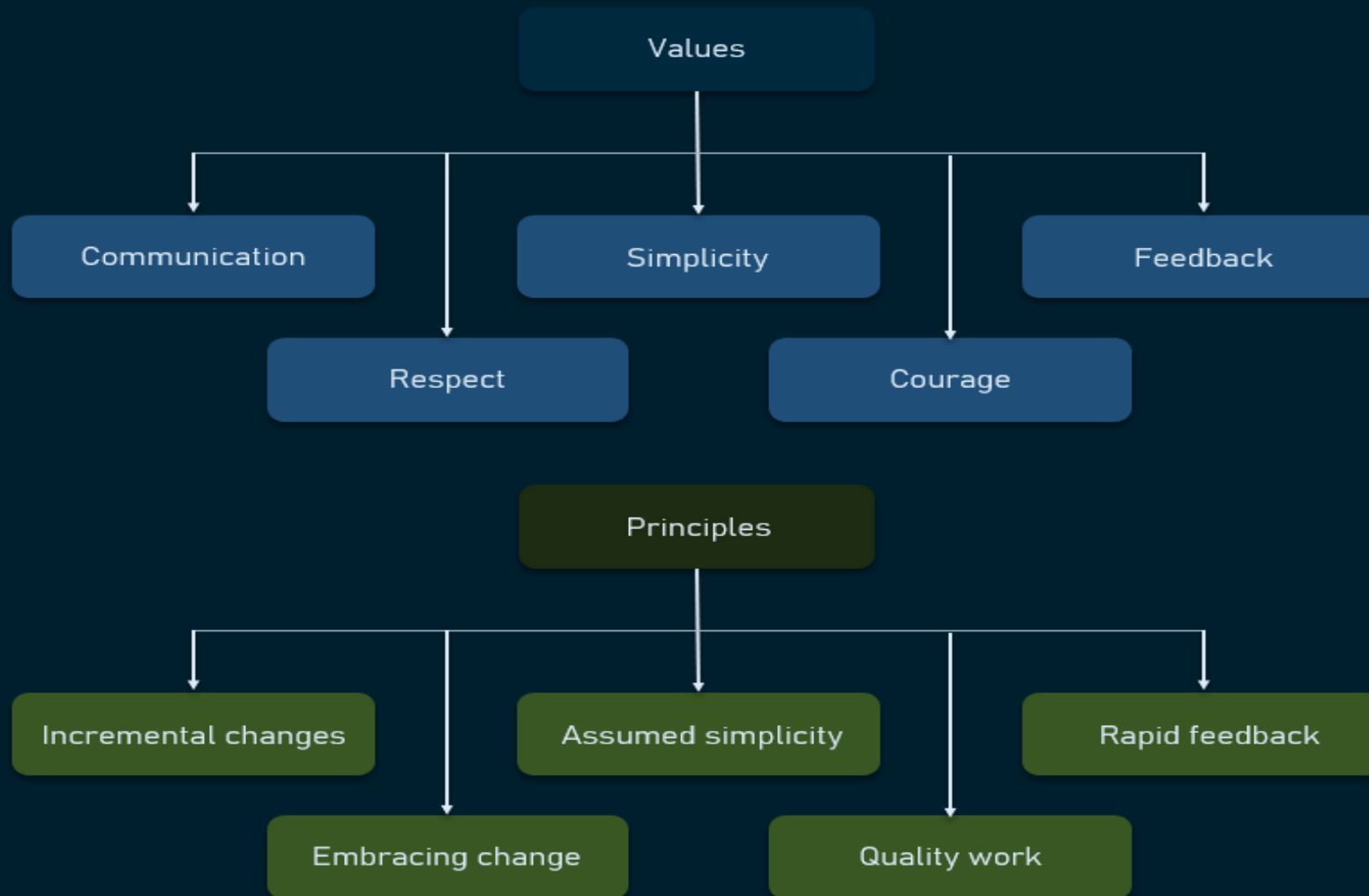
Sprint Retrospective

The team comes together to document and discuss what worked and what didn't work during the Sprint. Ideas generated are used to improve future Sprints.

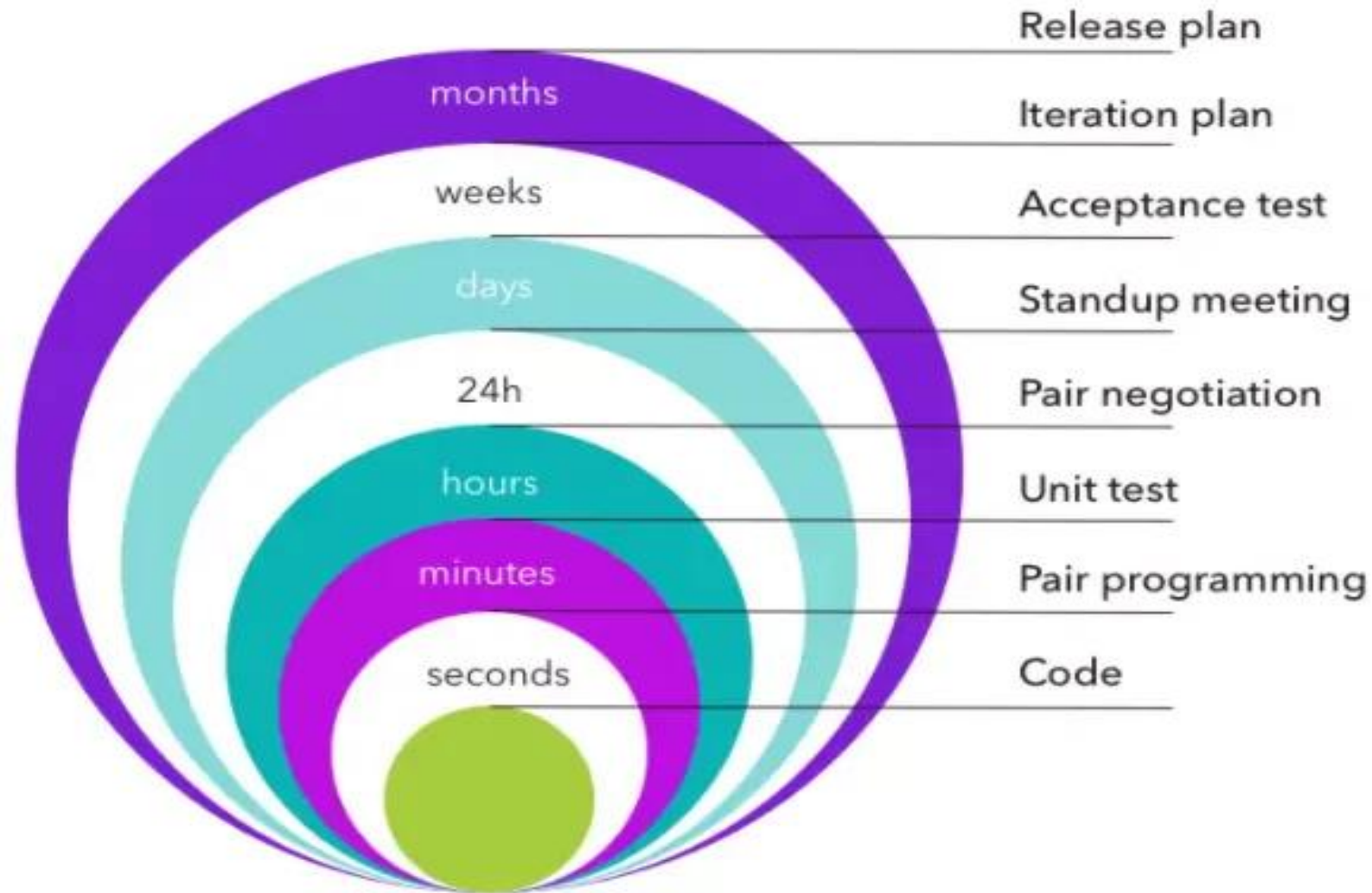
What is Extreme Programming?

Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software and higher quality of life for the development team. XP is the most specific of the agile frameworks regarding appropriate engineering practices for software development.

EXTREME PROGRAMMING VALUES AND PRINCIPLES



XP Feedback Loops



XP is built on a foundation of values that guide the team's work:

- Communication:**

Fostering open and honest communication between team members and with customers to ensure everyone is aligned.

- Simplicity:**

Focusing on simple, clean, and flexible designs that are easy to understand and maintain, rather than complex, anticipatory designs.

- Feedback:**

Obtaining continuous feedback through frequent testing and early delivery of working software to quickly identify and address issues.

- Courage:**

The willingness to express opinions, make difficult decisions, and take the necessary steps to improve the product.

- Respect:**

Valuing each team member's contributions and concerns, leading to better collaboration and problem-solving.

Key Practices

XP's values are supported by a set of specific engineering practices that are taken to an "extreme" level:

- Pair Programming:**

Two programmers work together at one workstation to write code, providing continuous code review and knowledge sharing.

- Test-Driven Development (TDD):**

Developers write automated tests before writing the actual code, ensuring the code meets the requirements and allowing for quick validation.

- Continuous Integration:**

Code is integrated frequently into a shared repository, and automated builds and tests are run to catch integration issues early.

- Simple Design:**

Continuously refactoring the system to keep the design simple and maintainable as requirements evolve.

- Frequent Releases:**

Delivering working software to the customer in short iterations, allowing for rapid feedback and early delivery of value.

- On-site Customer:**

Having a representative from the customer's team working closely with the development team to provide immediate feedback and clarify requirements.

Benefits of XP

- High-Quality Software:**

Practices like TDD and pair programming contribute to cleaner, more robust code.

- Adaptability:**

The focus on frequent releases and continuous feedback allows teams to quickly adapt to changing customer needs.

- Collaboration:**

XP promotes a collaborative environment with high customer involvement, leading to a better understanding of project goals.

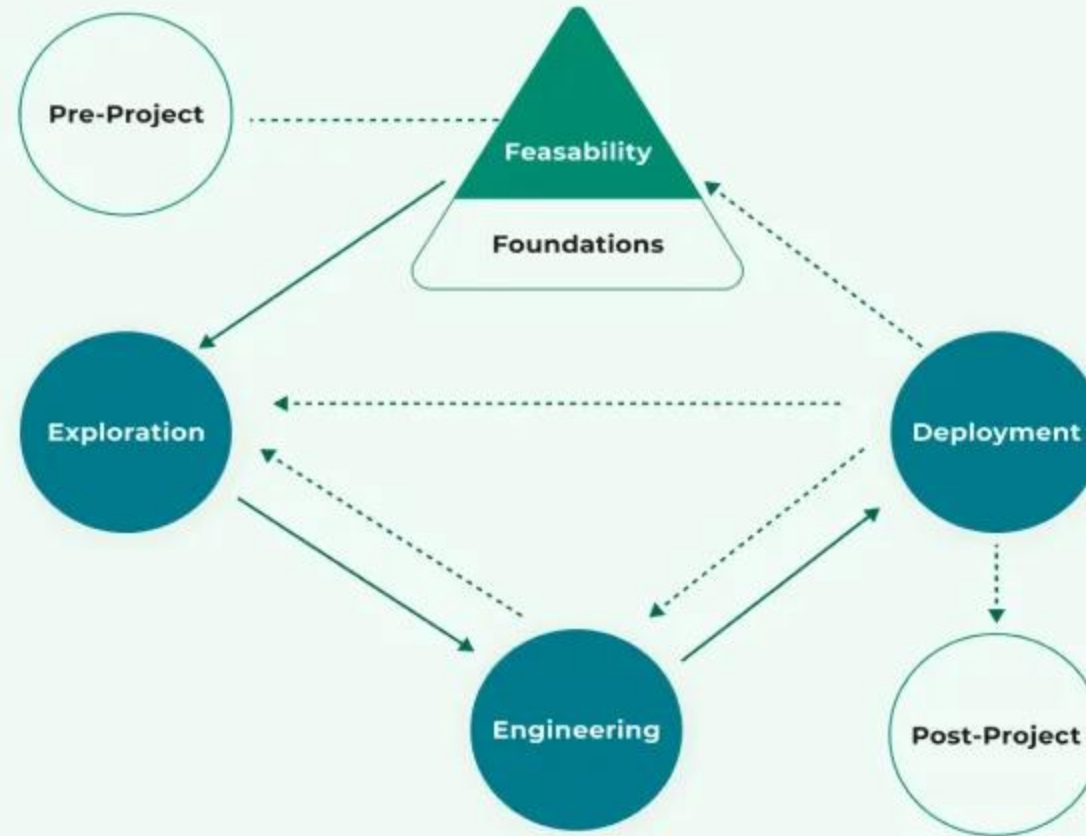
- Technical Excellence:**

Emphasizes strong engineering practices, resulting in sustainable development and higher technical quality.

DSDM(Dynamic Systems Development Method)

DSDM, or the Dynamic Systems Development Method, is an agile project management framework that provides structure to agile development. It uses an iterative and collaborative approach to build software and other products, focusing on delivering business value quickly. DSDM emphasizes discipline, teamwork, and clear communication to ensure projects stay aligned with business goals and are delivered on time.

Dynamic Systems Development



Key Aspects of DSDM

- Agile and Iterative:**

DSDM is an agile methodology that breaks projects into iterative cycles to ensure continuous involvement of stakeholders and quick delivery of working solutions.

- Focus on Business Value:**

The core principle is to deliver tangible benefits to the business early and often by focusing on the most critical features first.

- Collaboration:**

It promotes close collaboration between business and technical teams, fostering communication and shared responsibility for project success.

- Timeboxing:**

DSDM uses timeboxing to set firm deadlines for tasks, which helps to control costs and ensure that projects are delivered on schedule.

- Flexibility:**

While it provides discipline, DSDM allows for flexibility by enabling stakeholders to de-scope or postpone lower-priority requirements to protect the project's end date.

The DSDM Lifecycle

The DSDM lifecycle typically involves six phases:

- 1.Feasibility Study:** Establishes business needs and constraints.
- 2.Foundation:** Sets the groundwork for the project and identifies core requirements.
- 3.Evolutionary Development:** Involves iterative development of the solution.
- 4.Deployment:** The final phase where the solution is released.

Adaptive Software Development (ASD)

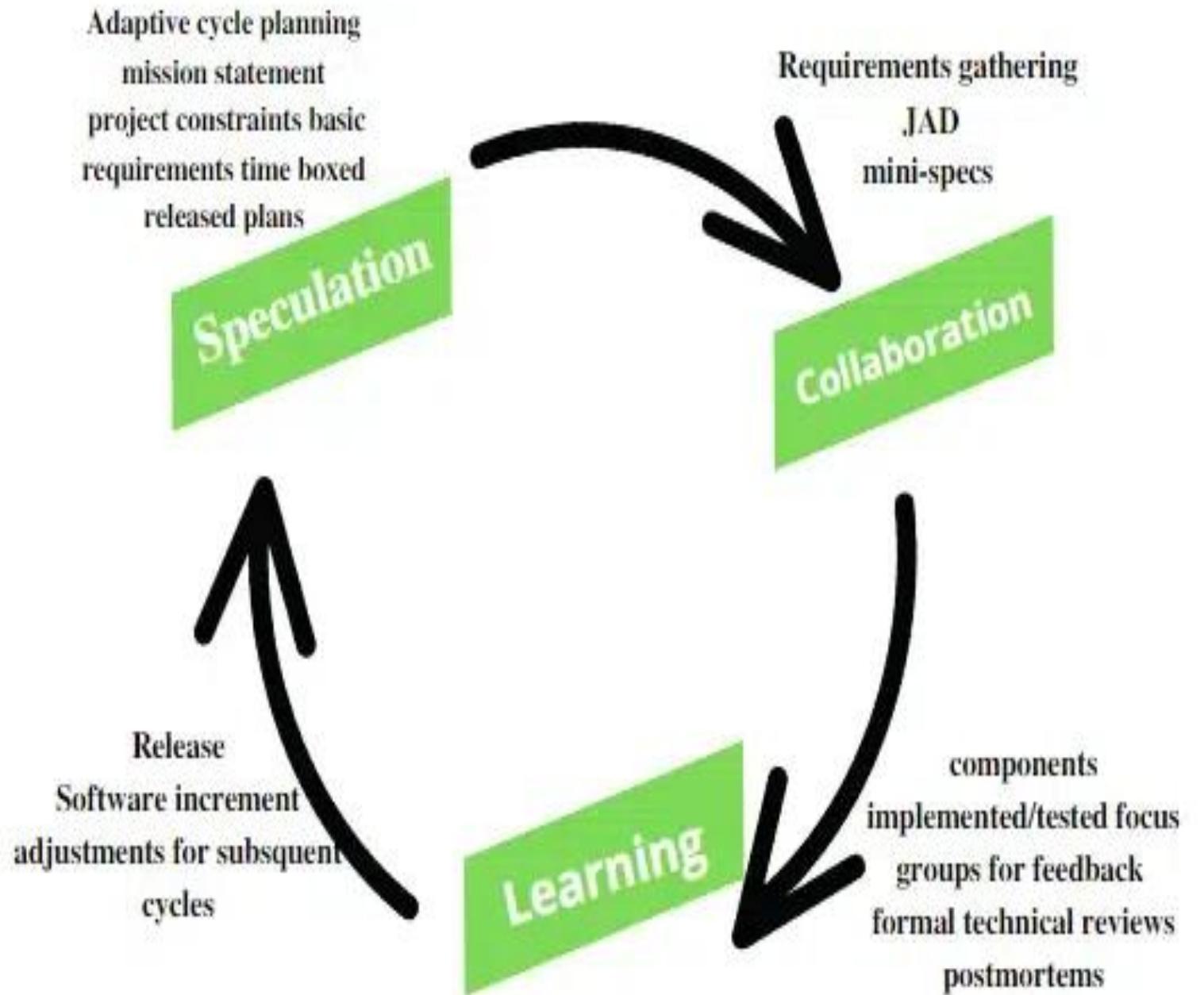
Adaptive Software Development (ASD) is a [software development methodology](#) that emphasizes adaptability to respond to changing requirements and environments. It focuses on continuous learning and is considered a part of agile software development.

- 1.Iterative Development:** ASD breaks down the project into small, manageable iterations, delivering incremental values.
- 2.Risk Management:** ASD involves identifying and addressing risks early in the development process.
- 3.Continuous Testing:** It integrates testing throughout the development process to ensure high-quality outcomes.
- 4.Decentralized Control:** It encourages decision-making at the team level instead of relying on top-down directives.
- 5.Customer Satisfaction:** It prioritizes customer satisfaction by delivering products that meet their needs and expectations.

Phases of ASD Process

ASD life cycle incorporates three phases namely:

- 1.Speculation
- 2.Collaboration
- 3.Learning



1. Speculation

During this phase project is initiated and planning is conducted. The project plan uses project initiation information like project requirements, user needs, customer mission statement, etc, to define a set of release cycles that the project wants.

2. Collaboration

It is the difficult part of ASD as it needs the workers to be motivated. It collaborates communication and teamwork but emphasizes individualism as individual creativity plays a major role in creative thinking. People working together must trust each other to

- Criticize without animosity,
- Assist without resentment,
- Work as hard as possible,
- Possession of skill set,
- Communicate problems to find effective solutions.

3. Learning

The workers may have an overestimate of their understanding of the technology which may not lead to the desired result. Learning helps the workers to increase their level of understanding of the project.

The learning process is in 3 ways:

- 1.Focus groups

- 2.Technical reviews

- 3.Project postmortem

ASD's overall emphasis on the dynamics of self-organizing teams, interpersonal collaboration, and individual and team learning yields software project teams that have a much higher likelihood of success.

The Crystal Model

The Crystal Model is a family of lightweight, adaptive agile frameworks for software development that prioritizes people and their interactions over strict processes and tools. Created by Alistair Cockburn, Crystal methods are color-coded to reflect project criticality (from Crystal Clear for small, non-critical projects to Crystal Red for larger, higher-risk ones). It empowers teams to tailor their practices, focusing on effective communication, frequent delivery, reflective improvement, and technical proficiency to suit the specific needs of each project.

Key Principles and Characteristics

- Focus on People:**

- Emphasizes human interaction, collaboration, and community as key success factors.

- Adaptability:**

- Each Crystal method is tailored to project size, criticality, and other factors, allowing teams to define their own solutions.

- Lightweight Documentation:**

- Minimizes documentation, management, and reporting in favor of transparency and efficient workflows.

- Continuous Improvement:**

- Promotes reflective improvement cycles and frequent feedback to adapt and enhance the development process.

- Technical Excellence:**

- Encourages features like automated tests, continuous integration, and configuration management for robust development.

Crystal Colors and Their Meanings

The different "colors" of Crystal represent variations designed for different project characteristics:

- Crystal Clear:** For very small, non-critical projects with few people.
- Crystal Yellow:** For teams of around 10-20 people.
- Crystal Orange:** For larger, more critical projects with 20-50 team members.
- Crystal Red:** For large projects with high criticality, often involving teams larger than 50 members.

How it Works

1. Assess Project Needs:

The team first evaluates the project's size, criticality, and priority.

2. Choose the Right Color:

Based on these factors, the appropriate Crystal method (color) is selected.

3. Tailor the Framework:

The team then adapts the selected Crystal framework, choosing practices that align with their specific workflow and project needs.

4. Iterative Development:

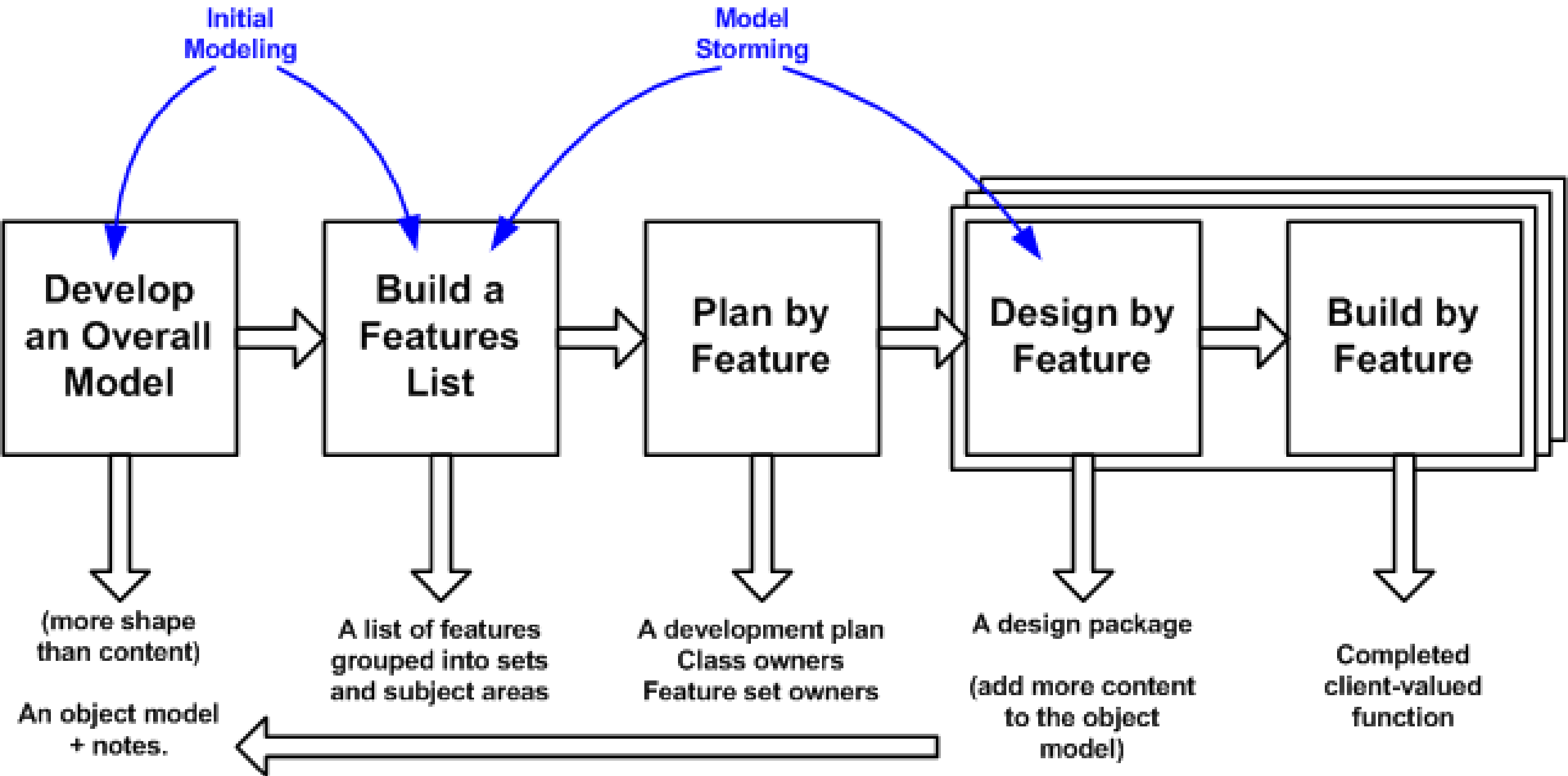
Development proceeds through iterations, with a focus on building and delivering functional software in increments.

5. Continuous Feedback and Improvement:

Regular communication and feedback loops are used to refine the process and ensure the software meets evolving requirements.

FDD(Feature-Driven Development)

FDD in software engineering stands for Feature-Driven Development. It's an agile software development methodology that focuses on delivering working software in small, client-valued increments called "features". FDD emphasizes collaboration, clear feature scoping, and iterative development to ensure user needs are met.



Here's a more detailed explanation:

Core Principles:

- Feature-centric:** Development is organized around delivering features, which are small, client-valued functions.
- Iterative and Incremental:** FDD uses short, iterative development cycles to build and refine features.
- Model-Driven:** FDD emphasizes building a domain model to guide development.
- Collaboration:** FDD promotes teamwork and communication through documentation and regular inspections.
- Progress Tracking:** FDD emphasizes regular status reporting and progress tracking at all levels.

Key Activities:

FDD is characterized by five key activities, performed iteratively:

- 1. Develop an Overall Model:** Establish a high-level understanding of the system and its domain.
- 2. Build a Features List:** Identify and define the specific features to be developed.
- 3. Plan by Feature:** Organize the development process around individual features.
- 4. Design by Feature:** Design and model the implementation of each feature.
- 5. Build by Feature:** Develop, test, and integrate the features.

Agile Modeling

Agile Modeling is a flexible, practice-based methodology for creating models and documentation in software development, emphasizing collaboration, iterative development, and quick feedback to support projects that are open to change. It involves lightweight, "low-tech, high-touch" techniques like whiteboarding to create models that are "just barely good enough" for their intended purpose, rather than overly complex or detailed documentation.

Core Principles & Practices

Agile Modeling (AM) is a collection of values, principles, and practices focused on effective modeling and documentation. Key aspects include:

- Collaboration:**

Involves customers and stakeholders in the modeling process to gather rapid feedback.

- Iterative Development:**

Creates models incrementally, fitting the process to the project's needs rather than a rigid, long-term plan.

- Lightweight Documentation:**

Focuses on creating models that are "just barely good enough," prioritizing understanding and value over excessive detail.

- Adaptability:**

Welcomes and incorporates changes to project requirements, making the modeling process flexible and responsive.

- Focus on Value:**

Ensures that the effort invested in creating and maintaining models provides positive value to the project.

Benefits

Applying Agile Modeling can lead to several advantages:

- Improved Quality:**

Early and continuous feedback helps to improve the quality of the developed system.

- Faster Time-to-Market:**

Iterative and incremental development allows for quicker delivery of functional software.

- Reduced Costs:**

By avoiding unnecessary activities and focusing on essential documentation, costs can be reduced.

- Flexibility:**

The methodology easily accommodates changes in project scope or requirements.

- Better Alignment:**

Close collaboration ensures that the data models and software align with evolving business requirements.

Assessment Model – CMMI (Capability Maturity Model Integration)

CMMI is a **process improvement framework** that assesses the maturity of an organization's software development process.

It helps measure **how well an organization develops software** and suggests improvements.

CMMI Level 1 – Initial

1. Characteristics

- Processes are **not defined or documented**.
- Work is done in an **ad-hoc (unplanned, unorganized)** manner.
- Success depends mainly on the **skills of individuals**, not on a standard process.
- There is **no consistency** — one project might succeed, another may fail.

Think of it like **cooking without a recipe**. Sometimes the dish comes out tasty if the cook is skilled, sometimes it's a disaster.

2. Example

Suppose a **startup company** is building a **Hospital Management System**.

- They do not prepare **requirements documents** or **design blueprints**.
- Programmers directly start coding what they think the hospital wants.
- There are **no testing standards** → developers test only on their own machines.
- Each programmer follows their **own coding style**.
- No project schedule is followed — features are added whenever someone feels like it.

Result → If programmers are **talented and hardworking**, the hospital software may work. If not, the project fails.

CMMI Level 2 – Managed

1. Characteristics

- At this stage, the organization has **basic project management practices**.
- Requirements are **defined, recorded, and tracked** → not just in people's minds.
- Projects are **planned with schedules, budgets, and responsibilities**.
- Progress is **monitored and controlled**.
- Still project-focused (each project may have its own way of working, not organization-wide).

Think of it as **writing down the recipe and following it step by step**, so even if the main cook is absent, someone else can prepare the dish.

CMMI Level 3 – Defined

1. Characteristics

- At this stage, the organization has **organization-wide standard processes** → not just project-level.
- These processes are **well-documented, standardized, and shared** across all projects.
- Projects don't reinvent methods → they **reuse** common processes, templates, and best practices.
- There is **training for staff** so that everyone knows how to follow the processes.
- The focus shifts from **project management** (Level 2) to **organization-wide process improvement**.

Analogy:

Instead of each cook having their own recipe, the **restaurant chain (organization)** has a **standard recipe book**.

So no matter which branch you visit, the taste is the same.

CMMI Level 4 – Quantitatively Managed

1. Characteristics

- At this stage, the organization not only follows **standard processes (Level 3)**, but also **measures and controls them using quantitative data (metrics)**.
- Decisions are based on **facts and statistics**, not just experience or intuition.
- Variations in process performance (like time taken, defects found, cost overruns) are **measured, tracked, and controlled**.
- Focus = **predictability** → we can forecast project outcomes with data.

Analogy:

In a restaurant chain, not only is there a standard recipe, but chefs now **measure every ingredient precisely** (200g flour, 100ml milk, etc.). The outcome (taste, quality) becomes **consistent and predictable**.

CMMI Level 5 – Optimizing

1. Characteristics

- At this stage, the organization has reached the **highest maturity level**.
- Focus is not just on following processes or measuring them, but on **continuously improving** them.
- Uses **innovation, feedback, and lessons learned** from past projects to refine practices.
- Encourages **new technologies, tools, and methods** for improvement.
- Problems are addressed **proactively**, not just reactively.

Analogy:

The restaurant chain not only has recipes and measurements, but also **keeps experimenting** with new dishes, healthier ingredients, or faster cooking techniques to stay ahead of competitors.

Software Development Life Cycle (SDLC)

SDLC is a systematic process for planning, creating, testing, deploying, and maintaining software. It ensures high-quality software that meets customer requirements.

Phases of SDLC

1. Requirement Analysis

- Understand *what* the client wants.
- Example: A hospital wants a **Hospital Management System** to manage patients, doctors, and billing.
- Deliverable → **SRS (Software Requirements Specification)**.

2. System Design

- Convert requirements into architecture and design.
- Example: Designing **ER diagrams, database schema**, UI design.
- Deliverable → **Design Documents**.

3. Implementation (Coding)

- Developers write the actual code using the chosen programming language.
- Example: Implementing patient registration in **Java + MySQL**.

1. Testing

- Ensure the system is bug-free and meets requirements.
- Example: Test if patient details are stored correctly.
- Types: Unit Testing, Integration Testing, System Testing, UAT.
- Deliverable → **Test Reports.**

2. Deployment

- Software is delivered to the client or deployed on the server.
- Example: Deploy hospital system on hospital's local server.

3. Maintenance

- Fix issues, add new features, improve performance.
- Example: Adding **online appointment booking** later.

Requirement Engineering (RE)

Requirement Engineering is the **systematic process of defining, documenting, and maintaining the requirements** for a software system.

It ensures that the **software meets user needs, business goals, and technical feasibility**.

Objectives of Requirement Engineering

- Understand **what the customer really needs**.
- Avoid **miscommunication** between stakeholders and developers.
- Provide a **clear, unambiguous specification** of requirements.
- Serve as a **basis for system design, coding, and testing**.
- Handle **changing requirements** systematically.

Characteristics of Good Requirements

A good requirement must be:

- **Correct** – represents stakeholder needs.
- **Unambiguous** – only one possible interpretation.
- **Complete** – covers all scenarios.
- **Consistent** – no contradictions.
- **Verifiable** – can be tested/validated.
- **Feasible** – technically and financially possible.
- **Traceable** – linked to its origin.

Modifiable – easy to change if needed

Requirement Engineering (Phases)

Requirement Engineering is often divided into **five major activities**:

(i) Requirement Elicitation (Gathering)

- Collecting requirements from stakeholders.
- Techniques:
 - Interviews
 - Questionnaires
 - Brainstorming
 - Observation
 - Workshops
 - Prototyping
 - Use Cases/User Stories

(ii) Requirement Analysis

- Understand, refine, and resolve conflicts among gathered requirements.
- Classify requirements:
 - **Functional requirements** (what system should do).
 - **Non-functional requirements** (performance, security, usability).
- Model requirements using UML, ER diagrams, etc.

(iii) Requirement Specification

- Documenting requirements clearly.
- Formats:
 - **SRS (Software Requirement Specification)** – IEEE 830 standard.
 - Use cases, user stories, diagrams.

(iv) Requirement Validation

- Check if requirements are correct, complete, and aligned with user needs.
- Techniques:
 - Reviews
 - Walkthroughs
 - Prototyping
 - Test-case generation

(v) Requirement Management

- Handle changes in requirements during development.
- Track requirement status using tools (JIRA, Trello, IBM DOORS).

Maintain **traceability** (requirement → design → code → test)

Tools for Requirement Engineering

- **JIRA** – Agile requirement tracking.
- **IBM Rational DOORS** – Requirement management.
- **RequisitePro** – Requirement documentation.
- **StarUML / Enterprise Architect** – Modeling requirements.

Trello, Asana – Task & requirement management

Challenges in Requirement Engineering

- **Ambiguity** in user requirements.
- **Changing requirements** due to evolving business needs.
- **Conflicts** among stakeholders.
- **Communication gap** between technical team and clients.

Over-specification or under-specification.

Types of Requirements

1. Functional Requirements (FRs)

- Define what the system should do.
- Describe functions, features, and services of the system.
- Usually expressed as input → process → output.

Examples:

- "System shall allow users to register with email and password."
- "ATM should dispense cash when a valid PIN is entered."
- "System must generate monthly sales reports."

2. **Non-Functional Requirements (NFRs)**

- Define how well the system performs (quality attributes).
- Constraints on performance, usability, reliability, etc.
- Critical for system acceptance.

Types of NFRs:

- Performance → Response time should be < 2 sec.
- Reliability → System should have 99.9% uptime.
- Security → Data must be encrypted with AES-256.
- Usability → System should be easy to use for non-technical staff.
- Scalability → Support up to 1 million users.

3. Domain Requirements

- Requirements imposed by the application domain (business/industry rules).
- Often come from standards, laws, or regulations.

Example:

- Banking software → "Interest calculation must follow RBI rules."
- Healthcare system → "Patient data storage must follow HIPAA regulations."

4. User Requirements

- High-level statements of what users expect from the system.
- Written in natural language, often for non-technical stakeholders.

Example:

- "The system should allow customers to track their order status."
- "The app should provide reminders for medicine intake."

5. System Requirements

- Detailed technical specifications derived from user requirements.
- Written in a formal structure (e.g., SRS document).
- Divided into functional + non-functional requirements at system level.

Example:

- "The system shall store order details in a relational database."

6. Business Requirements

- High-level goals of the organization.
- Explain *why* the system is being developed.

Example:

- "Reduce customer support costs by 20%."
- "Increase online sales by integrating digital payments."

7. Regulatory / Compliance Requirements

- Requirements imposed by government policies, industry standards, or legal obligations.

Example:

- "System must comply with GDPR data privacy rules."
- "Financial transactions must be logged for 7 years."

8.Interface Requirements

- Define interaction between system and external entities.
- Covers UI, hardware, APIs, and communication protocols.

Example:

- "System must provide REST API for third-party applications."
- "Mobile app should run on both Android and iOS."

9. Transition Requirements

- Requirements for moving from current system → new system.
- Temporary, valid only during migration.

Example:

- "Data must be migrated from Oracle to MySQL database."
- "Old system should run in parallel for 3 months."

10. Stakeholder Requirements

- Needs and expectations of stakeholders (customers, users, developers, managers).
- May be conflicting, require negotiation.

Example:

- Customer wants low cost → Developer wants maintainability → Manager wants faster delivery.

SRS

Software Requirements Specification (SRS)

SRS is a detailed written document that describes what a software system should do and the constraints under which it must operate.

It serves as a bridge between stakeholders (customers, end-users, managers) and developers (designers, programmers, testers).

Purpose of SRS

- Provide a clear understanding of what is to be built.
- Reduce ambiguity and misunderstandings.
- Serve as a reference document for design, coding, and testing.
- Act as a contract between customer and developer.
- Help with project estimation (cost, time, resources).
- Basis for acceptance testing.

Characteristics of a Good SRS

A good SRS should be:

- Correct → represents actual needs.
- Unambiguous → only one interpretation possible.
- Complete → covers all functionalities.
- Consistent → no contradictions.
- Verifiable → test cases can be derived.
- Modifiable → easy to change.

Traceable → each requirement linked to its source

Structure of an SRS (IEEE 830 Standard)

An SRS generally follows this structure:

1. Introduction

- Purpose of the system
- Scope of the project
- Definitions, acronyms, abbreviations
- References (documents, standards)
- Overview of the document

2. Overall Description

- Product perspective (relation to existing systems)
- Product functions (summary of major functions)
- User characteristics (technical background of users)
- Constraints (hardware, software, regulatory)

Assumptions and dependencies

3. Specific Requirements

- Functional Requirements (detailed system behaviors)
- Non-Functional Requirements (performance, usability, security, etc.)
- External Interface Requirements (user interface, hardware, software, communication protocols)
- System Features (detailed descriptions of system modules)

4. Appendices

- Glossary
- Supporting information
- References

Benefits of SRS

- For Developers → Provides a blueprint to build the system.
- For Testers → Acts as basis for test case design.
- For Customers → Ensures their requirements are captured.
- For Managers → Helps in tracking and project estimation.

7. Common Mistakes in SRS

- Writing requirements in ambiguous language ("system should be fast").
- Mixing requirements with design details.
- Incomplete requirements (missing error-handling).
- Ignoring non-functional requirements.

Software Requirement Specification (SRS)

For: Hospital Management System

Document Version: 1.0

Prepared by: [Your Name]

Date: [DD/MM/YYYY]

1. Introduction

1.1 Purpose

The purpose of this document is to describe the functional and non-functional requirements for the Hospital Management System (HMS). The system will manage patient records, doctor scheduling, billing, and pharmacy modules.

1.2 Scope

The HMS will provide a centralized platform for managing hospital operations.

- Patients can register and book appointments.
- Doctors can access patient history and prescribe medicines.
- Admin can manage staff, billing, and inventory.

The system reduces paperwork, increases efficiency, and improves patient care.

1.3 Definitions, Acronyms, and Abbreviations

- HMS: Hospital Management System
- GUI: Graphical User Interface
- DBMS: Database Management System

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specification
- User manuals of existing HMS systems

1.5 Overview

This document covers system requirements, external interfaces, functional requirements, and non-functional requirements.

2. Overall Description

2.1 Product Perspective

The HMS is a **standalone web-based application** with database support. It interacts with:

- Users (Patients, Doctors, Admins)
- Third-party payment gateways

2.2 Product Features

- Patient registration and history management
- Doctor appointment scheduling
- Pharmacy and billing management
- Reporting and analytics

2.3 User Classes and Characteristics

- Admin:** Full access to all modules
- Doctor:** Access to patient records, prescriptions
- Patient:** Limited access (appointments, prescriptions, bills)

2.4 Operating Environment

- Server: Windows/Linux
- Database: MySQL/PostgreSQL
- Frontend: Web (HTML/CSS/JS, React)
- Backend: Java/Python/Node.js

2.5 Design and Implementation Constraints

- Must comply with data privacy laws
- Works only with internet connectivity
- Database must support at least 1M records

2.6 Assumptions and Dependencies

- Users must have basic computer knowledge
- Hospital must have internet access

3. External Interface Requirements

3.1 User Interfaces

- Login/Registration page
- Dashboard for each user type
- Search functionality
- Printable reports

3.2 Hardware Interfaces

- Standard desktop/laptop/mobile device
- Minimum 4GB RAM

3.3 Software Interfaces

- Payment Gateway (e.g., Razorpay/PayPal API)
- Email/SMS Notification API

3.4 Communications Interfaces

- HTTPS protocol
- Secure authentication (OAuth2 / JWT)

4. System Features

4.1 Patient Management

Description: Manage patient profiles and history.

Functional Requirements:

- FR1: System shall allow patients to register.
- FR2: System shall store and retrieve medical history.

4.2 Appointment Scheduling

Description: Book and manage doctor appointments.

Functional Requirements:

- FR3: Patients can view available slots and book.
- FR4: Doctors can approve/reject appointments.

4.3 Billing & Pharmacy

Description: Generate bills and manage pharmacy stock.

Functional Requirements:

- FR5: System shall auto-generate bills after consultation.
- FR6: Pharmacy stock must update automatically.

5. Non-Functional Requirements

- Performance:** Must handle 200 concurrent users.
- Security:** Data encryption (AES-256). Role-based access control.
- Usability:** Intuitive GUI, <3 clicks to complete major tasks.
- Reliability:** 99.5% uptime required.
- Scalability:** Must support future integration with lab systems.

Thank You